INSTITUTO TECNOLÓGICO DE TIJUANA

SEMESTRE:
SEXTO SEMESTRE (ENERO - AGOSTO 2026)

CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:
Lenguaje de Interfaces

NOMBRE Y NÚMERO DE CONTROL DE LOS ALUMNOS:
Medina Lopez Emilio Ignacio - 23212010

NOMBRE DEL MAESTRO:
Rene Solis Reyes

FECHA DE ENTREGA:
16 de abril del 2026

Proyecto: Librería de Alto Rendimiento en ARM64 con Python, C y Assembly

Introducción

Este proyecto implementa una librería de alto rendimiento en arquitectura **ARM64 (AArch64)** utilizando una integración de tres niveles:

- Python como interfaz de usuario y pruebas
- C como capa de enlace (bridge)
- Assembly ARM64 como núcleo de optimización

El objetivo es analizar el rendimiento y comportamiento de llamadas entre lenguajes y evaluar cuándo el uso de ensamblador es justificado en sistemas modernos.

Arquitectura del sistema

```
Python (app.py)
  ↓ ctypes
C (bridge.c)
  ↓ linking
Assembly ARM64 (ops.s)
  ↓
libops.so
```

Estructura del proyecto

```
codigo/
├── src/
│   ├── app.py
│   ├── bridge.c
│   └── ops.s
├── build/
├── evidencias/
```

```
├── Makefile
├── README.md
└── reporte.pdf
```

Implementación

3.1 Assembly ARM64

Se implementaron funciones básicas optimizadas:

- suma
- resta
- multiplicación
- máximo y mínimo
- suma de arreglo
- conteo de pares
- producto punto

Ejemplo de instrucción clave:

add x0, x0, x1

Uso de registros:

- x0–x7: argumentos
- x0: retorno
- w registros: operaciones de 32 bits

3.2 C Bridge

El archivo bridge.c actúa como interfaz ABI entre Python y Assembly.

Ejemplo:

extern int add(int a, int b);

```
int c_add(int a, int b) {
  return add(a, b);
}
```

3.3 Python (ctypes)

Python carga la librería:

```
lib = ctypes.CDLL("build/libops.so")
```

Permite ejecutar funciones en C/ASM como si fueran nativas.

Pruebas y resultados

Se realizaron pruebas funcionales y benchmarking.

Ejemplo:

- Python puro: suma en loop
- C/ASM: llamada directa a librería

Análisis de rendimiento

¿Por qué Assembly puede ser más rápido?

- Elimina overhead de abstracción
- Control total de registros
- Menos instrucciones por operación
- Optimización manual de memoria

Costo de llamadas

Python → C → ASM

Cada salto implica:

- cambio de contexto
- manejo de stack
- conversión de tipos

Por eso no siempre conviene usar ASM.

Debugging avanzado (GDB)

Compilar con símbolos:

gcc -g -fPIC -c bridge.c

gcc -g -shared bridge.o ops.o -o libops.so

Iniciar GDB:

gdb python

Comandos útiles:

Breakpoints

break add

break main

Ejecutar

run app.py

Registros ARM64

info registers

Ver memoria

x/10x $x0

Step en assembly

stepi

nexti

Errores comunes

Segmentation Fault

- acceso inválido a memoria
- punteros mal pasados desde Python

Error de linkage

- símbolo no encontrado
- falta .global en ASM

Arquitectura incorrecta

- compilar en x86 en lugar de ARM64

Retos avanzados

Optimización de loops

- reducir instrucciones por iteración
- usar registros temporales eficientemente

Instrucciones condicionales

csel x0, x0, x1, gt

Acceso eficiente a memoria

- uso de offsets con LSL #2

Introducción a NEON

- vectorización SIMD
- procesamiento paralelo de datos

Conclusión

La integración de Python, C y Assembly ARM64 permite:

- alto rendimiento en operaciones críticas
- control de bajo nivel del hardware
- análisis de costos entre abstracción y velocidad

Sin embargo, el uso de Assembly debe reservarse para:

- optimización crítica
- sistemas embebidos
- operaciones intensivas en CPU

Sistemas embebidos

- uso en IoT
- control de sensores
- procesamiento en tiempo real